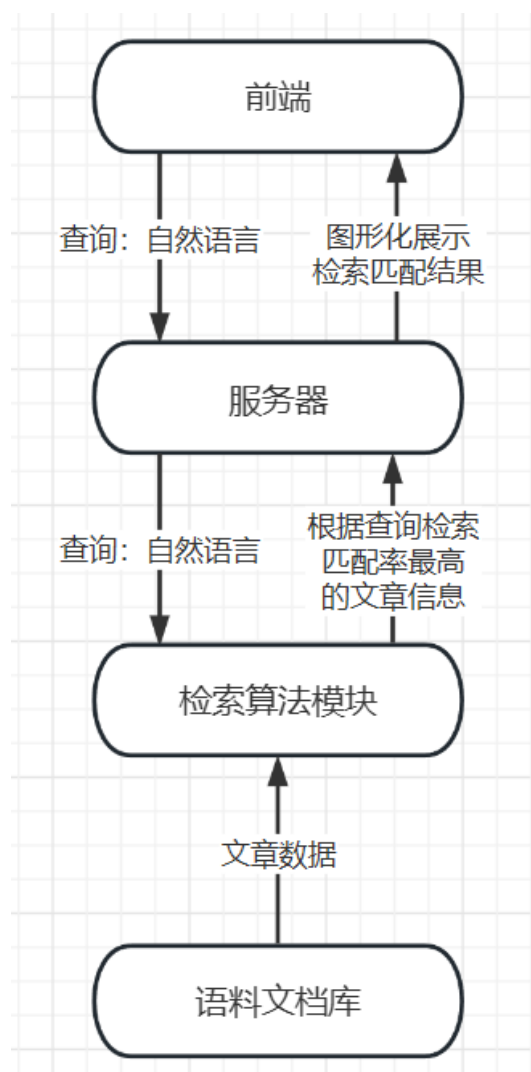


基于向量空间检索模型和TF-IDF的信息检索系统

系统结构与运行流程



- 总体介绍:

系统分为前端、服务器、检索算法三个模块。系统启动时，服务器控制检索算法模块加载语料文章内容，并对文章进行分词并建立倒排索引。之后系统启动前端，用户在前端页面查询框中输入查询的自然语言，前端将该查询发送服务器端，服务器调用检索算法模块中检索算法检索出内容匹配率最高的前几个文章，并将文章标题、url、匹配得分发送到前端。用户可在得到自己希望检索的文章后点击前往浏览内容。

语料的获取与存放

1. 从小说网站爬取小说文本，应选取章节数目较多的小说，下面以中文小说《斗罗大陆》为例：

爬取小说标题、内容rawdata，url，并对内容进行分词，将数据保存为json文件

爬取源码：

```
import os
import requests
import jieba
import time
from bs4 import BeautifulSoup
import json
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64;
x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57'
}

chapters = [] # 存储所有章节的数据
count = 1

def crawl(path, url, depth=1):
    global count
    if not os.path.exists(path):
```

```

        os.mkdir(path)
    if depth == 0:
        return

    print('-' * 30)
    print('Crawling:', url)
    if(count % 100 == 0): time.sleep(10)
    response = requests.get(url, headers=headers)
    if response.status_code == 200:
        print('Status code:', response.status_code)
        print('Content type:', response.headers['Content-
Type'])
        print('Encoding:', response.encoding)
        print('-' * 30)
        text = response.text.encode(response.encoding,
errors='ignore').decode('gb2312', errors='ignore')
        soup = BeautifulSoup(text, 'lxml')

        chapter_data = {} # 存储当前章节的数据

        # 获取标题
        main_div = soup.find('div', {'id': 'main'})
        title_element = main_div.find('h1') if main_div
else None
        title = title_element.text.strip() if
title_element else ''
        chapter_data['headline'] = title

        # 获取URL
        chapter_data['url'] = url

        # 获取内容
        content = soup.get_text().strip()
        chapter_data['content'] = content
        chapter_data['content_seg'] = jieba.lcut(content)

        # 将当前章节的数据添加到列表中
        chapters.append(chapter_data)

```

```

# 递归爬取下一级章节
for link in soup.find_all('dd'):
    for a in link.find_all('a'):
        if a.has_attr('href'):
            if a['href'].startswith('http'):
                crawl(path, a['href'], depth-1)
            else:
                crawl(path, url + a['href'],
depth-1)

        if len(content):
            print('Text length:', len(content))
            with open(os.path.join(path, 'chapter-
'+str(count)+'json'), 'w+', encoding='utf-8') as f:
                json.dump(chapter_data, f,
ensure_ascii=False, indent=4)
            print('Saved to:', os.path.join(path,
'chapter-'+str(count)+'json'))
            count += 1
        else:
            print('Error:', response.status_code)

```

2. 对原始数据进行预处理，去除分词结果中无关字符、标点符号等，并保存为一个json文件以减少IO次数

处理源码：

```

import json
import os
import re

json_data_list = []
output_file_path =
'../doucments/DouLuo_Json/douluo1.json'

def remove_invalid_words(words):
    valid_words = []

```

```

    for word in words:
        if not re.match(r'^[\u4e00-\u9fa5]+$', word):
            continue
        valid_words.append(word)
    return valid_words

for file_name in os.listdir('../documents/DouLuo_Json'):
    with open(os.path.join('../documents/DouLuo_Json',
file_name), 'r', encoding='utf-8') as file:
        if(file_name[0:7] != 'chapter'):
            print(file_name)
            continue

        json_data = json.load(file)
        words = json_data['content_seg']
        valid_words = remove_invalid_words(words)
        json_data["content_seg"] = valid_words
        json_data_list.append(json_data)
with open(output_file_path, 'w', encoding='utf-8') as
output_file:
    json.dump(json_data_list, output_file,
ensure_ascii=False, indent=4)

```

文章数据保存json格式

```

[
    {
        "标题(headline)": ,
        "地址(url)": 1,
        "文章原内容": ,
        "分词结果": [] ,
    },
]

```

检索算法基本原理及优化

相关概念

■ 倒排索引

该系统中，倒排索引通过将文档集合中的每个单词映射到包含该单词的所有文档的字典，值对应文档中该单词的出现频率，从而支持通过词快速检索出对应文档和词频。

结构：{'word': {doc_id1: freq1, doc_id2: freq2, ...}, ...}

■ 向量空间模型

在向量空间模型中，查询和文档都表示为所有词构成的n维向量 $(q_1, q_2, \dots, q_n)$ 和 $(d_1, d_2, \dots, d_n)$ ， d_i 和 q_i 是该词在查询或某一文档中的出现频率。通过计算查询向量和文档向量之间的相关性，来检索匹配查询的文档。

■ TF-IDF

$$\text{词频}(TF) = \frac{\text{词在文章中出现次数}}{\text{文章中总词数}}$$

$$\text{逆文档频率}(IDF) = \log\left(\frac{\text{语料库中文档总数}}{\text{包含该词的文档数} + 1}\right)$$

$$TF - IDF = \text{词频}(TF) \times \text{逆文档频率}(IDF)$$

- 理解：TF-IDF是用于衡量一个词对于一个文档集合中某一文档重要性的统计方法，通过词频 (TF)和逆文档频率(IDF)的组合来衡量词的重要程度。

- 词频(TF)越高，说明一个词在文档中的出现次数越多，那么它对于文档的重要性就越高。
- 逆文档频率表示一个词在整个文档集合中的稀有程度，如果在整个文档集合中包含该词的文档数越少，那么该词的代表性就越强，用于描述区分文档的能力也越强。

检索准确率评估

从语料库中没一个文档中随机抽取一定数量的词组成查询向量，并使用算法查询，若检索出文档相关度最高的文档为原文档，则定义为检索准确，否则为不准确。

$$\text{检索准确率} = \frac{\text{检索准确的文档数}}{\text{参与检索的总文档数}} \times 100\%$$

仅使用向量夹角余弦计算相关度

余弦相关度公式：

$$\cos(\vec{q}, \vec{d}) = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}}$$

通过评估方法评估检索准确率：

检索章节数：689

检索匹配章节数：462

检索准确率：67.05370101596516 %

经程序计算准确率保持在67%~69%

令文档向量中每个词权重 $w_i = d_i * idf$

由于逆文档频率(idf)表示一个词在整个文档集合中的稀有程度，如果在整个文档集合中包含该词的文档数越少，那么该词的代表性就越强，其用于描述区分文档的能力也越强，所以令文档向量中每个词的权重 d_i (即该词在该文档中出现的频率tf) * idf 可以提高更稀有的词在对文档的代表权重，提高文档匹配准确率。

如某一个人名只在小说中某一章节中出现，那么这个人名对这一章节的文档的代表性就很强。

通过评估方法评估检索准确率：

```
=====
检索章节数：689
检索匹配章节数：504
检索准确率：73.14949201741655 %
=====
```

经程序计算准确率保持在73%~75%

令文档向量中每个词的权重 $w_i = (1 + \log(d_i)) * idf$

由于文档中某些词可能多次出现，而某些重要的包含很大信息量的词仅出现少量次数，这些少量出现的词往往对于匹配查询的内容更加重要。

如某一个人A在小说某一章中做了一次事情X，词A在这一章中很多次出现，在其它章节中也出现很多次，而词X只在这一章中出现了一次，那么对于查询"A做了X事件"X词更加重要，对于该文档也更具代表性。

为了减少极端值的权重和对查询的影响，令词的权重 $w_i = (1 + \log(d_i)) * idf$ ，从而使词的影响度随词频增大而增长地更加平缓，避免掩盖掉出现频率较少的词的影响度。

通过评估方法评估检索准确率：

```
=====
检索章节数：689
检索匹配章节数：688
检索准确率：99.85486211901306 %
=====
```

经程序计算准确率保持在99%~100%

其它：

使用停用词表，可以过滤敏感词，以及过滤掉信息量较低的词，如“了”，“的”，“是”，“不是”，“着”等在每个文档中都会大量出现的词，使其无法影响检索准确率。

算法源码：

```
import os
import math
from collections import defaultdict
from cut import cut_sentence
import json

class RetrievalModel:
    def __init__(self, path):
        self.path = path
        self.index = defaultdict(dict)
        self.doc_length = {} # doc_length 用于存储每个文档的长度
        self.total_length = 0 # 所有文档总长度 所有文件长度之和
        # 是指所有文件中所有词的长度之和？
        self.docs = [] # docs 用于存储所有文档文件名
        self.stop_words = set() # stop_words 用于存储停用词表
        self.load_stop_words() # 载入停用词表
        # self.build_index() # 建立倒排索引
        # self.build_index_with_json() # 建立倒排索引 using
        # json files
        self.build_index_with_ljson() # 建立倒排索引 using l
        # json file
        self.compute_lengths() # 计算每个文档的长度

    def load_stop_words(self): # 载入停用词表
        file_path = os.path.join(os.path.dirname(__file__),
        'stop_words.txt')
        with open(file_path, 'r', encoding='utf-8') as f:
            for line in f:
                self.stop_words.add(line.strip())
```

```

def build_index_with_json(self): # 建立倒排索引
    for doc_id, file in enumerate(os.listdir(self.path)):
# enumerate()函数用于将一个可遍历的数据对象组合为一个索引序列，同时
# 列出数据和数据下标
        self.docs.append(file)
        with open(os.path.join(self.path, file), 'r',
encoding='utf-8') as f:
            data = json.load(f)
            tokens = data["content_seg"]
            for token in tokens:
                if token not in self.stop_words:
                    self.index[token][doc_id] =
self.index[token].get(doc_id, 0) + 1

def build_index_with_ljson(self): # 建立倒排索引
    with open(os.path.join(self.path, 'douluo.json'),
'r', encoding='utf-8') as f:
        data = json.load(f)
        for doc_id in range(len(data)):
            self.docs.append(data[doc_id]["headline"])
            tokens = data[doc_id]["content_seg"]
            for token in tokens:
                if token not in self.stop_words:
                    self.index[token][doc_id] =
self.index[token].get(doc_id, 0) + 1

# {'word': docname{doc_id: freq, doc_id: freq, ...}, ...}
def build_index(self): # 建立倒排索引
    for doc_id, file in enumerate(os.listdir(self.path)):
# enumerate()函数用于将一个可遍历的数据对象组合为一个索引序列，同时
# 列出数据和数据下标
        self.docs.append(file)
        with open(os.path.join(self.path, file), 'r',
encoding='utf-8') as f:
            for line in f:
                tokens = line.strip().split()
                for token in tokens:
                    if token not in self.stop_words:

```

```

        self.index[token][doc_id] =
self.index[token].get(doc_id, 0) + 1

    def compute_lengths(self): # 计算文档长度
        for doc_id in range(len(self.docs)):
            length = 0
            for term, freq in self.index.items(): # get
(term, {doc_id: freq, doc_id: freq, ...})
                tf = freq.get(doc_id, 0) # term frequency
                idf = math.log(len(self.docs) /
len(self.index[term])) # 计算单词的逆文档频率 = log(文档总数 /
包含该单词的文档数)
                if tf > 0:
                    # length += (1 + math.log(tf)) # 加1避免
tf为1时tf_weight为0
                    length += ((1 + math.log(tf)) * idf * (1
+ math.log(tf)) * idf) # 先取对数再乘逆文档频率
                self.doc_length[doc_id] = math.sqrt(length)
                self.total_length += math.sqrt(length) # 将文档长
度加到所有文档总长度上

    def calculate_query_vector(self, query): # 计算查询向量
        query_vector = defaultdict(int)
        for term in query.strip().split():
            if term not in self.stop_words:
                query_vector[term] += 1
            length = math.sqrt(sum(map(lambda x: x*x,
query_vector.values())))) # 计算查询向量的长度(模) map()返回一个
迭代器
        for term, freq in query_vector.items():
            query_vector[term] = freq / length # 单词的出现次
数 / length = 单词的权重
        return query_vector # 返回查询向量

    def calculate_score(self, query, words_counter): # 计算文
档得分
        query_vector = self.calculate_query_vector(query) #
得到查询向量

```

```

        scores = defaultdict(float)
        for term, freq in query_vector.items(): # (term,
term_freq_in_query)
            if term in self.index:
                idf = math.log(len(self.docs) /
len(self.index[term])) # 计算单词的逆文档频率 = log(文档总数 /
包含该单词的文档数)
                for doc_id, tf in self.index[term].items(): #
(doc_id, term_freq_in_doc)
                    words_counter[doc_id][term] = tf # 记录每
个文档中每个单词出线次数
                    tf_weight = 1 + math.log(tf) # 单词在文档
中的权重 = 1 + log(单词频率)
                    scores[doc_id] += freq * tf_weight * idf
/ self.doc_length[doc_id] # 文档得分 = 查询向量中单词权重 * 文档
中单词权重 * 逆文档频率 / 文档长度
                for doc_id, score in scores.items():
                    scores[doc_id] *= self.total_length # 文档得分 =
文档得分 * 所有文档总长度
                sorted_scores = sorted(scores.items(), key=lambda x:
x[1], reverse=True) # 对第二个元素排序
                return sorted_scores

    def search(self, query, num_results=5): # 查询
        query = cut_sentence(query) # 对查询分词
        words_counter = defaultdict(dict)
        print('匹配分词:', query)
        results = self.calculate_score(query, words_counter)
[:num_results] # num_results个文档

        with open(os.path.join(self.path, 'douluo.json'),
'r', encoding='utf-8') as f:
            data = json.load(f)
            ret = []
            for doc_id, score in results:
                record = {'title': [], 'url': [],
'matchRate': []}
                print(f.name)

```

```
record['title'] = data[doc_id]["headline"]
record['url'] = data[doc_id]["url"]
record['matchRate'] = score
ret.append(record)
print('Document:', self.docs[doc_id])
print('Score:', score)
print(words_counter[doc_id])
    # with open(os.path.join(self.path,
self.docs[doc_id]), 'r', encoding='utf-8') as f: # 打开文档
    #     print('Content:', f.readline().strip()) #
打印文档内容的第一行
    print('---' * 20)
return ret
```